Capítulo 7: Herramientas de Git

Tutorial de Git basado en Pro Git (2.ª ed.) · Chacon y Straub · Elaborado por Ricardo · UNIAGRARIA

Introducción

Este capítulo reúne herramientas avanzadas que permiten seleccionar revisiones, preparar commits con precisión, guardar trabajo temporal, buscar, reescribir la historia, depurar y gestionar proyectos complejos.

7.1 Revisión por selección

```
git show 1c002dd              # hash completo o abreviado
git rev-parse --short HEAD
git log --oneline --abbrev-commit
git show HEAD~2                  # abuelo del commit actual
git show HEAD^2                  # segundo padre de un merge
git reflog                       # historial de movimientos de HEAD
git show HEAD@{5}
```

Rangos de commits

```
git log main..experimento     # en experimento pero no en main
git log origin/main..HEAD        # lo que aún no se ha subido
git log main...experimento       # en uno u otro, pero no en ambos
git log --left-right main...experimento
```

7.2 Organización interactiva

```
git add -i       # menú interactivo
git add -p       # elegir fragmentos (hunks) a preparar
git restore -p   # descartar fragmentos selectivamente
```

Permite dividir los cambios de un mismo archivo en varios commits lógicos.

7.3 Guardado rápido y Limpieza

```
git stash                 # guarda cambios y deja el directorio limpio
git stash -u              # incluye archivos sin seguimiento
git stash list
git stash apply           # reaplica sin borrar
git stash pop             # reaplica y borra
git stash drop
git stash branch rama-nueva

git clean -n              # simula qué se borraría
git clean -f -d           # borra archivos y carpetas sin seguimiento
```

7.4 Firmando tu trabajo

```
gpg --list-keys
git config --global user.signingkey <ID-clave>
git tag -s v1.0 -m "Versión firmada"
git tag -v v1.0
```

```
git commit -S -m "Commit firmado"
git log --show-signature -1
```

7.5 Buscando

```
git grep -n "funcion"               # busca en el árbol de trabajo
git grep --count "funcion"
git log -S"funcion" --oneline           # commits que añaden o quitan esa cadena
git log -L :funcion:archivo.c            # historia de una función
```

7.6 Reescribiendo la Historia

```
git commit --amend                  # modifica el último commit
git rebase -i HEAD~3                # reordenar, editar, combinar, borrar commits
# En el editor: pick, reword, edit, squash, fixup, drop

git filter-branch --tree-filter 'rm -f passwords.txt' HEAD   # reescritura masiva
(obsoleto: usar git filter-repo)
```

> **Nota:** Reescribir la historia cambia los hashes de los commits afectados; no lo hagas con commits ya publicados y compartidos.

7.7 Reiniciar Desmitificado (reset)

Git gestiona tres árboles: HEAD (último commit), el índice (siguiente commit) y el directorio de trabajo. git reset actúa sobre ellos según la opción:

- --soft: mueve HEAD (y la rama) pero deja el índice y el directorio de trabajo.
- --mixed (por defecto): mueve HEAD y actualiza el índice; deja intacto el directorio de trabajo.
- --hard: mueve HEAD, actualiza el índice y sobrescribe el directorio de trabajo (peligroso).

```
git reset --soft HEAD~1
git reset HEAD~1
git reset --hard HEAD~1
git reset archivo.txt          # equivale a quitarlo del área de preparación
```

7.8 Fusión Avanzada

```
git merge --abort                   # cancela una fusión con conflictos
git merge -Xours rama             # favorece nuestros cambios en conflictos
git merge -Xignore-space-change rama
git checkout --ours archivo
git checkout --theirs archivo
git diff --ours
git log --merge -p                # historial relevante al conflicto
git revert -m 1 <hash-del-merge>  # deshace una fusión
```

7.9 Rerere

"Reuse recorded resolution" memoriza cómo se resolvió un conflicto y lo reaplica automáticamente cuando vuelve a aparecer (útil en rebases repetidos).

```
git config --global rerere.enabled true
```

7.10 Haciendo debug con Git

```
git blame -L 12,22 archivo.txt     # quién cambió cada línea
git blame -C archivo.txt

# Búsqueda binaria del commit que introdujo un error
git bisect start
git bisect bad              # el commit actual tiene el error
git bisect good v1.0        # esta versión funcionaba
# Git va saltando entre commits; se marca cada uno good/bad
git bisect reset
```

7.11 Submódulos

Permiten incluir otro repositorio como subdirectorio, fijado a un commit concreto.

```
git submodule add https://github.com/usuario/libreria.git libs/libreria
git clone --recurse-submodules <url>
git submodule update --init --recursive
git submodule update --remote
```

7.12 a 7.14 Agrupaciones, Replace y credenciales

```
# Bundle: empaquetar un repositorio en un archivo para transportarlo sin red
git bundle create repo.bundle HEAD main
git clone repo.bundle copia

# Replace: sustituir un objeto por otro sin reescribir la historia
git replace <hash-antiguo> <hash-nuevo>

# Credenciales
git config --global credential.helper cache     # en memoria 15 min
git config --global credential.helper store     # en texto plano
git config --global credential.helper manager   # administrador del sistema
(Windows/macOS)
```

Resumen

Git ofrece herramientas para seleccionar revisiones con precisión, componer commits por fragmentos, apartar trabajo con stash, buscar, depurar con blame y bisect, reescribir la historia con rebase interactivo y reset, resolver conflictos de forma más cómoda, y organizar proyectos con submódulos y bundles.

Referencia

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/es/v2 — Capítulo 7.